

AI IN ACTION - Day 22 - Track 1

Automated Evaluators

Khi nào nên automate · Code-based vs LLM-judge · Common mistakes

Instructor: Mai Anh Nguyen (Blue)

Vấn đề mới: scale

Manual review không thể scale

~100

traces mỗi tuần

- ✓ Hiểu được patterns
- ✓ Manual review khả thi

Build quá sớm →

Mất thời gian đo sai thứ

100k

production traces

- ✗ Không thể review hết
- ✗ Không thể chậm mỗi release

Build sai →

False confidence — system fail âm thầm

Không phải vấn đề nào cũng cần eval

Chỉ generalization gap mới là ứng viên tốt để tự động hóa

Loại lỗi	Khi nào xảy ra	Việc cần làm
Specification gap Thiếu đặc tả	Prompt chưa nói rõ yêu cầu, nên model làm sai. <i>Email drafter không tạo subject line vì prompt chưa yêu cầu.</i>	Sửa prompt Chưa cần viết eval
Architectural issue Thiếu năng lực hệ thống	Thiếu tool, integration, hoặc model không đủ capability. <i>Calendar API chưa connect — dù prompt rõ cũng không làm được.</i>	Engineering fix Chưa cần viết eval
Generalization gap Thiếu tính nhất quán	Prompt rõ, hệ thống có lúc làm đúng — nhưng không nhất quán. <i>Email drafter đôi khi cá nhân hóa đúng, nhưng chỉ ở case rất rõ ràng.</i>	Viết automated eval ✓ Đây là candidate phù hợp

Khi nào và vì sao nên viết automated evals

1

Ship nhanh hơn

Thay vì tranh luận "có tốt hơn không" — chạy eval, nhìn vào số liệu, ra quyết định.

2

Phát hiện quality giảm theo thời gian

Hệ thống từng ổn vẫn có thể kém đi khi gặp thêm edge cases từ production.

3

Phát hiện regression sau mỗi thay đổi

Sửa prompt, đổi model, thêm tool — luôn có rủi ro làm hỏng thứ trước đây đúng.

4

Tạo đòn bẩy cải thiện có hệ thống

Khi hành vi đã đo rõ, team cải thiện được mà không phải đoán mò từ đầu.

Automated evals không thay thế hiểu biết — chúng giải phóng team để tập trung vào những quyết định thật sự quan trọng.

Chọn cách đánh giá: Code-based hay LLM-as-Judge?

Nếu có thể kiểm tra bằng code — hãy dùng code trước

CODE-BASED — MẶC ĐỊNH ƯU TIÊN

- Nhanh hơn
- Rẻ hơn
- Ổn định
- Dễ gắn critical path
- Dễ debug

CODE LÀM TỐT

- Có đúng format không
- Có thiếu trường bắt buộc không
- Có gọi đúng tool không
- Có lộ thông tin cấm không
- Có trả lời sai schema không

LLM-AS-JUDGE — KHI CODE KHÔNG ĐỦ

Dùng khi tiêu chí phụ thuộc vào sắc thái ngôn ngữ và ngữ cảnh — không thể viết thành rule rõ ràng.

CODE KHÔNG LÀM TỐT

- Câu trả lời có hữu ích với đúng người dùng không?
- Explanation có hợp lý và nhất quán không?
- Cách diễn đạt có đủ thuyết phục không?
- Output có đúng brand voice không?

Code-based Evaluation

Viết deterministic checks chạy tự động trên mọi prompt change —
nền tảng không thể thương lượng của eval suite

Default path: nhanh hơn · rẻ hơn · reproducible 100%

ANATOMY CỦA MỘT CODE-BASED EVAL

Python · Deterministic · Không gọi LLM

① INPUT

```
def check(output_str, user_input):
```

② LOGIC

```
    if "Subject:" not in output_str:
```

③ RESULT + REASON

```
        return False, "Missing subject line"
    return True, None
```

3 phần của một code-based eval

① Input — lấy đúng phần cần kiểm tra

Output string, JSON, tool calls — hoặc kết hợp cả user input khi cần kiểm tra theo ngữ cảnh.

② Logic — rule đủ rõ để máy quyết định

Condition, pattern match, schema validation — nếu viết được "phải có X" hay "đúng format Z" → dùng code.

③ Result — pass/fail + reason string

Reason string không phải "có thì tốt" — khi fail ở trace 38/50, team phải biết ngay tại sao, không cần mở lại trace để đoán.

Category Checking — Support Triage Agent

Agent phải chọn đúng 1 trong 3 labels: Technical · Billing · Feature Request

```
VALID_CATEGORIES = {"Technical", "Billing", "Feature Request"}
```

```
def check_category_label(output: str) -> dict:
```

```
    found = [cat for cat in VALID_CATEGORIES if cat in output]
```

```
    if len(found) == 1:
```

```
        return {"pass": True, "reason": f"Single valid: {found[0]}"}  
    elif len(found) == 0:
```

```
        return {"pass": False, "reason": "No valid category found"}  
    else:
```

```
        return {"pass": False, "reason": f"Multiple labels:  
        {found}"}  
    else:
```

Danh sách labels được phép

Chỉ 3 giá trị exact string. "billing issue" hay "tech problem" không được tính — chỉ đúng chữ này.

Tìm bao nhiêu label xuất hiện trong output

Quét qua output string, gom lại tất cả labels tìm thấy. Kết quả là một list — rỗng, 1 phần tử, hoặc nhiều hơn.

3 trường hợp → 1 kết quả rõ ràng

pass Tìm thấy đúng 1 → ghi rõ label nào

fail Không có → "No valid category found"

fail Nhiều hơn 1 → liệt kê cụ thể để debug

4 nơi nên bắt đầu với Code-based Evals

Output có đúng cấu trúc không? structure / format	Đủ field, đúng format, đúng schema, trong giới hạn cho phép. Quan trọng khi output đi tiếp vào UI, database hoặc downstream system.
Output có đủ phần bắt buộc không? presence / coverage	Có nhắc đúng product name, ticket ID, required policy, source, next step? Bắt những output nghe ổn nhưng chưa đủ để user hành động.
Agent có đi đúng quy trình không? tool-call success / sequencing	Gọi đúng tool, đúng thứ tự, đúng parameter. Nhiều lỗi không nằm ở câu trả lời cuối mà nằm ở cách agent thực thi workflow.
Chỉ số vận hành có trong ngưỡng không? threshold checks	Latency, cost, token count, confidence score có vượt ngưỡng không? Trả lời đúng nhưng chậm hơn hoặc đắt hơn nhiều vẫn là regression.

Output phải dùng được · nội dung phải đủ ý · quy trình phải đúng · chi phí phải trong ngưỡng

Case Study: 3 kiểm tra tự động cho Support Triage Agent

KIỂM TRA 1

Câu trả lời có dùng đúng nhãn chuẩn không?

Phải chọn đúng một trong ba nhãn. Diễn đạt lại sẽ làm hỏng hệ thống phân loại phía sau.

✓ Đúng

Danh mục: Billing
Khách hàng khiếu nại hóa đơn tháng 3.

✗ Sai — không tìm thấy nhãn hợp lệ

Đây có vẻ là vấn đề liên quan đến thanh toán.

KIỂM TRA 2

Agent có tự bịa ra thông tin không có trong yêu cầu không?

So sánh mã ticket trong câu trả lời với mã trong yêu cầu ban đầu.

✓ Đúng

Yêu cầu có: TKT-00421
Câu trả lời chỉ nhắc: TKT-00421

✗ Sai — bịa ra mã TKT-00422

Yêu cầu có: TKT-00421
Câu trả lời nhắc: TKT-00422

KIỂM TRA 3

Thời gian phản hồi có còn trong ngưỡng chấp nhận được không?

Câu trả lời đúng nhưng chậm hơn nhiều vẫn là lỗi cần bắt.

✓ Đúng

Thời gian: 1.1 giây
Ngưỡng tối đa: 2.0 giây ✓

✗ Sai — vượt ngưỡng 2.0 giây

Thời gian: 2.4 giây
Sau khi cập nhật prompt

Cả 3 pass → mới xem xét release

Đây là mức sàn tối thiểu — những câu hỏi tinh tế hơn vẫn cần người đọc và đánh giá.

Đọc kết quả eval trên dataset

- Pass rate tổng là bao nhiêu?**
So với phiên bản production
- Failures tập trung ở đâu?**
Nhóm input nào đang fail nhiều?
- Lỗi có đi cùng nhau không?**
Row nào fail nhiều evals cùng lúc?
- Kết quả có bất thường không?**
Pass rate = 0% hoặc quá thấp

Tín hiệu tốt

- Pass rate bằng hoặc cao hơn baseline
- Failures rải đều — không có nhóm nào nổi bật
- Failures phân tán, không có row nào fail nhiều evals
- Kết quả nằm trong range hợp lý, khớp expectation

Cần xem lại

- Pass rate tụt đáng kể dù spot check trông ổn
- 80%+ failures từ cùng 1 loại input → failure mode cụ thể
- Cùng row fail 2+ evals → cụm input khó, không phải lỗi rời rạc
- Pass rate = 0% → kiểm tra eval trước!**
Regex sai, field sai — đừng vội điều tra agent

Đưa Code Evals vào quy trình release

NGUYÊN TẮC 1

Eval là release gate, không phải kiểm tra tùy hứng

Mỗi prompt change hoặc model upgrade chạy lại eval tự động.
Mọi thay đổi đi qua cùng một tiêu chuẩn — không chờ đến sát release.

Pass → Ship

Fail → Dừng

NGUYÊN TẮC 2

Chốt ngưỡng trước khi bắt đầu iterate

Quyết định sau khi thấy số → không còn là tiêu chuẩn mà là thương lượng.
Viết ngưỡng ra trước: nhấn >90%, 0 lỗi bịa nghiêm trọng, latency <2s.

"78% cũng ổn mà" → không phải threshold, là thương lượng

CHỌN CÔNG CỤ

Code eval hay LLM Judge?

→ **Code eval**

Có thể viết thành Python function ~10 dòng.
Rule rõ ràng: must include X, must not violate Y.

→ **LLM Judge**

Cần hiểu intent, đánh giá tone, reasoning tinh tế — không viết được thành rule.

LLM Judge Calibration

Hiệu chỉnh bộ chấm LLM Judge

AI Evals cho Product Manager

Vì sao calibration là bước cốt lõi

Pass rate chỉ cho biết đánh giá đang nghĩ gì

Thấy 85% pass rate không có nghĩa là đã đo được chất lượng thật sự.

Calibration mới cho biết đánh giá có đúng không

So kết quả LLM Đánh giá với đánh giá của người có chuyên môn.

Chưa calibrate còn nguy hiểm hơn không có

Tạo cảm giác chắc chắn sai — đang dùng hệ thống chưa kiểm chứng để ra quyết định ship.

VÍ DỤ: 10 TRACES

✓

✓

✓

✓

✓

✓

✗

✗

✗

✗

Người có chuyên môn đã chấm: 6 output TỐT · 4 output XẤU

TRONG 6 OUTPUT TỐT

LLM Đánh giá đồng ý đúng mấy lần?

✓

✓

✓

✓

✓

✗

5/6 = 83%

1 output tốt bị nhầm là xấu

TRONG 4 OUTPUT XẤU

LLM Đánh giá phát hiện đúng mấy lần?

✗

✗

✗

✓

3/4 = 75%

1 output xấu bị nhầm là tốt

Khi cả hai cùng cao → đánh giá đủ tin cậy. Khi một thấp → biết rõ lệch ở đâu để chỉnh.

Nhìn xem đánh giá đang lệch ở đâu

Calibration giúp biết đánh giá sai theo hướng nào — không chỉ biết "chưa tốt"

	Thật sự TỐT	Thật sự XẤU
Đánh giá nói TỐT	✓ Đúng Nhận ra đúng output tốt	⚠ Cho qua lỗi Nguy hiểm nhất Team tưởng ổn — lỗi thật vẫn lọt production
Đánh giá nói XẤU	⚠ Chặn nhầm Tổn thời gian Team đi sửa thứ không phải vấn đề thật	✓ Đúng Bắt đúng output xấu

Uncalibrated judges thường quá dễ dãi — xu hướng chấm tích cực hơn mức nên có, trừ khi prompt được thiết kế rõ để buộc khắt khe với failure.

6 bước calibration

- 1

Chọn 50-100 traces đại diện

Case dễ lẫn khó, mơ hồ
- 2

Expert gắn nhãn Pass/Fail

Đúng theo tiêu chí đánh giá đang đo
- 3

Chạy đánh giá — lưu verdict + reasoning

Kèm human label để đối chiếu
- 4

So sánh chỗ bất đồng với expert ★

Bước quan trọng nhất — thấy đánh giá lệch theo kiểu nào
- 5

Sửa prompt theo pattern sai

Đổi ít một — biết rõ cái nào có tác dụng
- 6

Kiểm tra trên tập riêng → baseline

Chưa ổn → quay lại bước 4

MỤC TIÊU

Không phải để đánh giá "nghe có vẻ hợp lý hơn"

Calibration là để biết đánh giá có đang chấm đủ gần với chuẩn của team hay chưa.

Có một judge mà team dám tin khi ra quyết định.

Khi nào LLM Judge đã chạm trần?

Nhận ra thời điểm này quan trọng — tránh tốn thời gian tối ưu thứ không còn cải thiện được

DẤU HIỆU 1

Model không thật sự hiểu domain

Reasoning strings cho thấy model thiếu nền tảng. Ví dụ: đánh giá legal citation, medical recommendation — model không có năng lực ở domain đó.

Vấn đề là model capacity — không phải prompt

DẤU HIỆU 2

Thêm prompt mà chất lượng gần như không nhích

Effort tăng nhưng improvement hầu như bằng không qua nhiều vòng. Đã rút gần hết giá trị có thể lấy từ model này.

Đã chạm điểm trần với model này

DẤU HIỆU 3

Ngay cả humans cũng chưa thống nhất

Tiêu chí còn quá mơ hồ. Khi con người chưa thống nhất pass/fail, mong judge chấm ổn định là không thực tế.

Làm chặt lại definition of quality trước

Vấn đề không còn nằm ở cách viết prompt — mà ở giới hạn của chính model hoặc tiêu chí đang được đánh giá.

Khi automation không chạm quality bar

Phân vai rõ giữa 3 lớp — không bỏ đo lường

LỚP 1 — LUÔN BẬT

Code-based
Evals

Structure, format, safety checks — automation đo tốt. Chạy trên mọi thay đổi.

- format
- schema
- safety

LỚP 2 — ĐÃ CALIBRATE

LLM Judge

Tiêu chí đã calibrate đủ tốt — semantic quality ở quy mô lớn.

- relevance
- grounding
- quality

LỚP 3 — FALLBACK

Human review

Final gate cho dimension automation chưa chạm. Sample có chủ đích: output cao nhất + thấp nhất.

Cần threshold rõ — không phải "có vẻ ổn"

Fallback tạm thời — mục tiêu dài hạn: LLM Judge tốt hơn hoặc đơn giản hóa tiêu chí

2 NGUYÊN TẮC KHI DỪNG HUMAN REVIEW

1. Sample có chủ đích — đừng review tất cả

Review toàn bộ sẽ quá tải. Lấy mẫu đúng chỗ đáng quan tâm:

- Output chấm cao nhất
- Output chấm thấp nhất

2. Đặt threshold rõ ràng

Human review cũng phải là hệ thống ra quyết định:

- bao nhiêu reviewers
- review bao nhiêu outputs
- bao nhiêu % phải đạt

⚙️ Codebase

- Schema, enum, format
- Permission, DB match
- Regex: UUID, PII, token
- Latency, cost, CI regression

Chạy đầu tiên và độc lập — không cần rubric hay human. Bắt lỗi chắc chắn trước khi chuyển sang human hoặc LLM.

1 Human review — định nghĩa "good"

- Đọc 10–30 trace đa dạng, gắn nhãn ✓ / ~ / ✗
- Ghi failure mode, severity, expected behavior
- Chọn golden outputs làm chuẩn

↓ Output: labeled cases + failure patterns

2 Tạo rubric + calibration set

- Viết tiêu chí pass/fail rõ ràng, có ví dụ tốt/xấu
- Tạo calibration set: 50–100 case có human label
- Đây là bước bắt buộc trước khi dùng LLM judge

↓ Điều kiện: rubric ổn định + human labels

3 LLM judge — scale những gì human đã định nghĩa

- Đưa rubric + examples vào judge prompt
- Đo precision/recall theo từng failure mode
- Audit định kỳ với human — không set-and-forget

✗ Không dùng LLM judge khi chưa có rubric

✗ Không bỏ human hoàn toàn ở high-stakes

✗ Không dùng code để chấm judgment

Công việc của PM là biến những cảm nhận mơ hồ về chất lượng thành các tiêu chí đánh giá rõ ràng

Human định nghĩa "tốt" trước — rồi mới có thể dạy LLM nhận ra "tốt" là gì